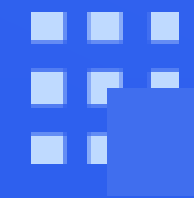


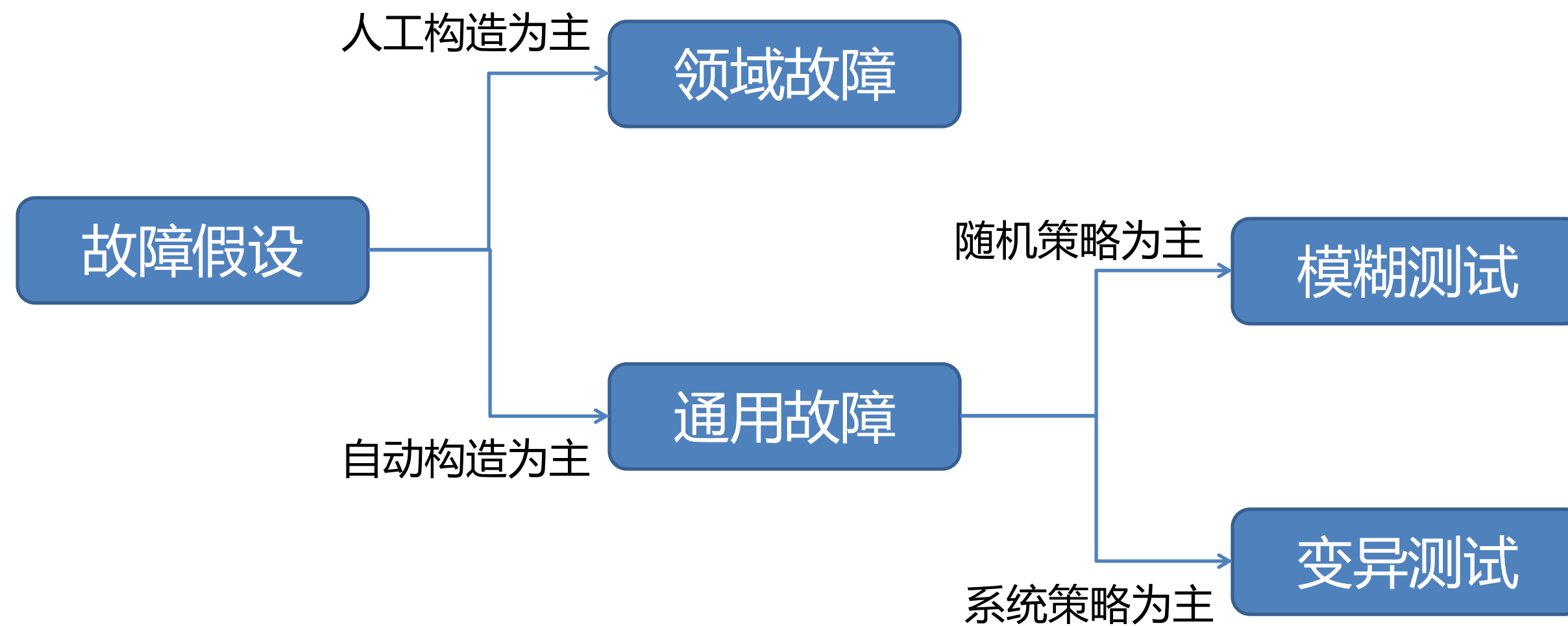


软件测试

变异分析基本概念



故障假设基本策略



变异故障假设测试概述

- 变异故障假设测试，也称为变异测试或变异分析，是一种对测试集的有效性和充分性进行评估的技术。
- 在变异分析的指导下，测试人员可以评价测试集或测试方法的缺陷检测能力，并辅助构建更强的测试集。
- 变异分析一定程度理论上揭示了常见的缺陷，即使这些特定类型的缺陷在分析的程序中不存在，也与缺陷检测能力相关。
- 变异分析通过将微小代码语法修改以检查测试集是否能够检测代码缺陷。

变异缺陷假设 ■ 变异分析基本概念

变异分析的基本假设

1978年，变异分析奠基人 Richard DeMillo 提出了两个重要基本假设。

熟练程序员假设

- 熟练程序员编写的有缺陷代码与正确代码非常接近。
- 意义：为变异测试的简单代码修改注入策略提供基础。

耦合效应假设

- 如果测试能检测单一缺陷，则也能检测耦合的复杂缺陷。
- 意义：为变异测试中优先考虑单一变异提供了理论基础。

变异体

定义 4.3: 变异体

变异体 $\mathcal{P}'$ 是原程序 $\mathcal{P}$ 进行符合语法规则的微小代码修改的程序版本。

- 微小代码修改：缺陷可通过小修改修复。
- 符合语法规则：确保变异体可以编译运行。
- 预期缺陷检测：期待测试 t 对于变异体 ρ' 和原程序 ρ 行为不一致。

初始程序

```
for (i=0; i<10; i++){  
    //to do....  
}
```

变异体二

```
for (i=0; i<=10; i++){  
    //to do....  
}
```

变异体一

```
for (i=1; i<10; i++){  
    //to do....  
}
```

变异体三

```
for (i=0; i<10; i--){  
    //to do....  
}
```

思考：（1）变异体一、二、三的差异？（2）变异体一、二、三的叠加？

变异体检测

定义 4.4: 变异体检测

若 t 使得 $\mathcal{P}'(t) \neq \mathcal{P}'(t)$, 则称测试 t 检测 (也称杀死) 变异体 $\mathcal{P}'$ 。



存活变异体: 如果 t 通过, 则称变异体 ρ' 对于 t 是存活的。



测试集 T : 如果存在 $t \in T$ 杀死变异体 ρ' 则称 T 杀死 ρ' 。

原
程
序

```
for (i=0; i<10; i++){  
    //to do....  
}
```

变
异
一

```
for (i=1; i<10; i++){  
    //to do....  
}
```

变
异
二

```
for (i=0; i<=10; i++){  
    //to do....  
}
```

变
异
三

```
for (i=0; i<10; i--){  
    //to do....  
}
```

思考: (1) 构造测试 t 使得变异体存活? (2) 构造测试 t 能杀死变异体?

变异分数

定义 4.5: 变异分数

给定变异体 $\mathcal{P}'_1, \dots, \mathcal{P}'_n$, 测试集 T 的变异杀死率, 也称为变异分数, 定义为

$$\text{变异分数} = \frac{\text{杀死的}\mathcal{P}'\text{个数}}{n} \quad (4.4)$$



直观理解: T 杀死的变异体越多, 测试质量越高。



主要作用: 变异分数是评估测试能力的重要度量。

讨论与思考: 变异分数依赖变异体, 同时也依赖测试集。

等价变异体

定义 4.6: 等价变异体

如果不存在 $t \in T$ 能杀死 $\mathcal{P}'$, 则称对于 T , $\mathcal{P}'$ 是 $\mathcal{P}$ 的等价变异体。



完全等价变异体: 若对于任意 T , ρ' 都是等价变异体, 则直接称 ρ' 是等价变异体。



行为特点: 语法不等价但语义等价, 即对于任意输入 t , ρ 和 ρ' 输出行为相同。

```
if (x!=5){  
    if(x>5){  
        //to do....  
    }  
}
```

```
if (x!=5){  
    if(x>=5){  
        //to do....  
    }  
}
```

思考: 实践中, 难以判定完全等价变异体, 因此常常用足够大的 T 来做充分性判定。

变异算子

定义 4.7: 变异算子

符合程序语法规则前提下，变异算子定义了生成变异体的变异规则。



- **值变异：**改变参数或参数的值。

例如，原程序 $x = 5$ ，变异程序为 $x = 10$ 。

- **语句变异：**增加、删除、替换语句，或更改语句顺序。

例如，原程序 $total = x - y$ ，变异程序为 $total = x + y$ 。

- **决策变异：**变逻辑或算术运算符。

例如，原程序 $if(x > y)$ ，变异程序为 $if(x < y)$ 。

变异分析练习 Triangle

```
def triangle(a, b, c):
    if not (1 <= a <= 100 and 1 <= b <= 100 and 1 <= c <= 100):
        print("无效输入值")
    elif not (a + b > c and b + c > a and c + a > b):
        print("无效三角形")
    elif a == b == c:
        print("等边三角形")
    elif a == b or b == c or c == a:
        print("等腰三角形")
    else:
        print("普通三角形")
```

简单尝试决策变异，并分析不同程序语言对变异分析的影响。

```
#include <stdio.h>
void triangle ( int a, int b, int c) {
    if (!(1 <= a && a <= 100 && 1 <= b && b <= 100 && 1 <= c && c <= 100)) {
        printf ( "无效输入值\n");
    } else if (!( a + b > c && b + c > a && c + a > b)) {
        printf ( "无效三角形\n");
    } else if (a == b && b == c) {
        printf ( "等边三角形\n");
    } else if (a == b || b == c || c == a) {
        printf ( "等腰三角形\n");
    } else {
        printf ( "普通三角形\n");
    }
}
```

变异分析练习 NextDay

```
def LeapYear(year):  
    if year % 400 == 0:  
        return 1  
    if year % 100 == 0:  
        return 0  
    if year % 4 == 0:  
        return 1  
    else:  
        return 0
```

```
def LeapYear(year):  
    if year % 4 == 0:  
        if year % 400 == 0:  
            return 1  
        elif year % 100 == 0:  
            return 0  
        else:  
            return 1  
    else:  
        return 0
```

综合尝试各类变异，并分析不同程序结构
对变异分析的影响。

```
def next_day(year, month, day):  
    tomorrow_month, tomorrow_day, tomorrow_year = month, day, year  
    if month in [1, 3, 5, 7, 8, 10]:  
        if day < 31:  
            tomorrow_day = day + 1  
        else:  
            tomorrow_day = 1  
            tomorrow_month = month + 1  
    .....  
    .....  
    elif month == 2:  
        if day < 28:  
            tomorrow_day = day + 1  
        elif day == 28:  
            if leap_year(year) == 1:  
                tomorrow_day = 29  
            else:  
                tomorrow_day = 1  
                tomorrow_month = 3  
        elif day == 29:  
            tomorrow_day = 1  
            tomorrow_month = 3  
    return (tomorrow_year, tomorrow_month, tomorrow_day)
```

弱杀死与强杀死

- 弱杀死：变异体的中间状态与原程序不同。
与之对应的称之为弱变异分析。
- 强杀死：变异体的输出结果与原程序不同。
与之对应的称之为强变异分析，也就是传统意义的变异分析。
- **输入 $X=[4,5,3]$, $L=3$, 分析弱杀死和强杀死**
- **输入 $X=[0,5,3]$, $L=3$, 分析弱杀死和强杀死**
- 思考：弱变异分析的优缺点？**

```
void MeanVar(double X[], int L) {  
    double mean, sum;  
  
    sum = 0;  
  
    for (int i = 1; i < L; i++) { // i的初始值应为0  
        sum += X[i];  
    }  
  
    mean = sum / L;  
  
    printf("Mean: %f\n", mean);  
}
```

系统级的变异分析

- 接口变异
- 功能变异
- 性能变异
-

复杂领域的变异分析

- 面向对象程序
- 区块链系统
- 人工智能系统
-

回顾与讨论

变异分析

两个假设

- 熟练程序员假设
- 耦合效应假设

基本概念

- 变异体
- 变异分数
- 变异算子
- 等价变异

领域拓展

- 系统级变异
- 面向对象变异
- 智能系统变异